

Linear Statistical Models - W7 GA

Utkarsh Sahu ~ 21F1001107

```
# W7Q1a
```

```
"
```

```
(a) Read the dataset as a data frame in R.
```

```
"
```

```
df <- read.table(file = "21F1001107_7/data/forbes.txt", header = TRUE)
```

```
View(df)
```

(row)	Temp	Pressure	Lpres
1	194.5	20.79	131.79
2	194.3	20.79	131.79
3	197.9	22.4	135.02
4	198.4	22.67	135.55
5	199.4	23.15	136.46
6	199.9	23.35	136.83
7	200.9	23.89	137.82
8	201.1	23.99	138
9	201.4	24.02	138.06
10	201.3	24.01	138.04
11	203.6	25.14	140.04
12	204.6	26.57	142.44
13	209.5	28.49	145.47
14	208.6	27.76	144.34
15	210.7	29.04	146.3
16	211.9	29.88	147.54
17	212.2	30.06	147.8

```
# W7Q1b
```

```
"
```

```
Using the 'ggplot' in R, plot the scatter plot between the 'Temp' and 'Pressure'.
Based on the obtained plot comment on the relationship between two variables.
```

```
"
```

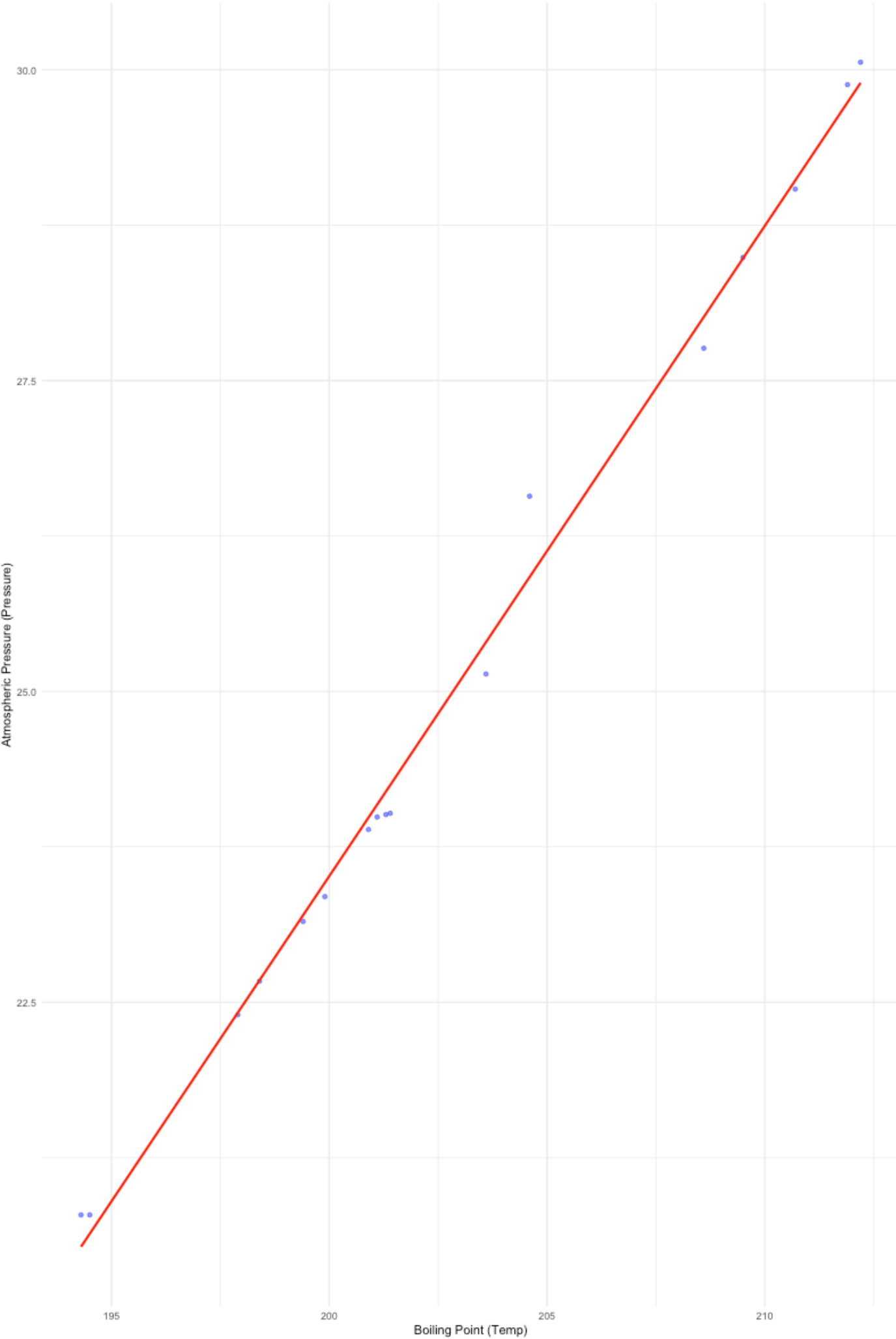
```
library(ggplot2)
```

```
ggplot(df, aes(x = Temp, y = Pressure)) +
  geom_point(
```

```
    color = "blue",
    alpha = 0.5
) +
geom_smooth(
  method = "lm",
  se = FALSE,
  color = "red"
) +
labs(
  title = "Boiling Point (°F) vs. Atmospheric Pressure (Inches Hg)",
  x = "Boiling Point (Temp)",
  y = "Atmospheric Pressure (Pressure)"
) +
theme_minimal()
```

ans_w7q1b <- "Temperature and Pressure have a Linear Relationship
with +ve correlation as visible from the scatterplot with the trendline."

Boiling Point (°F) vs. Atmospheric Pressure (Inches Hg)



W7Q1c

"

Obtain the least square estimates using normal equations and lm() function in R for the model:

$$L_{\text{pres}} = \beta_0 + \beta_1 * \text{Temp} + \varepsilon$$

where, $\varepsilon \sim N(0, \sigma^2)$.

Compare the least square estimates computed using the above-mentioned two methods.

"

Method 1: Normal Equations

y <- df\$Lpres

X <- cbind(1, df\$Temp)

Calculate $\beta_{\text{hat}} = (X'X)^{-1} * X'y$

beta_hat_normal <- solve(t(X) %*% X) %*% t(X) %*% y

cat("Estimates from Normal Equations:\n")

print(beta_hat_normal)

Method 2: lm() function

model_1 <- lm(Lpres ~ Temp, data = df)

beta_hat_lm <- coef(model_1)

cat("\nEstimates from lm() function:\n")

print(beta_hat_lm)

comment <- "

The intercept and slope, B0 and B1, are coming same from both the methods.

"

```
r$> # Method 1: Normal Equations
y <- df$Lpres
X <- cbind(1, df$Temp)

# Calculate beta_hat = (X'X)^-1 * X'y
beta_hat_normal <- solve(t(X) %*% X) %*% t(X) %*% y

cat("Estimates from Normal Equations:\n")
print(beta_hat_normal)
Estimates from Normal Equations:
      [,1]
[1,] -42.1377793
[2,]  0.8954937

r$>
```

```
r$> # Method 2: lm() function
model_1 <- lm(Lpres ~ Temp, data = df)
beta_hat_lm <- coef(model_1)

cat("\nEstimates from lm() function:\n")
print(beta_hat_lm)
```

```
Estimates from lm() function:
(Intercept)      Temp
-42.1377793    0.8954937
```

```
r$>
```

W7Q1d

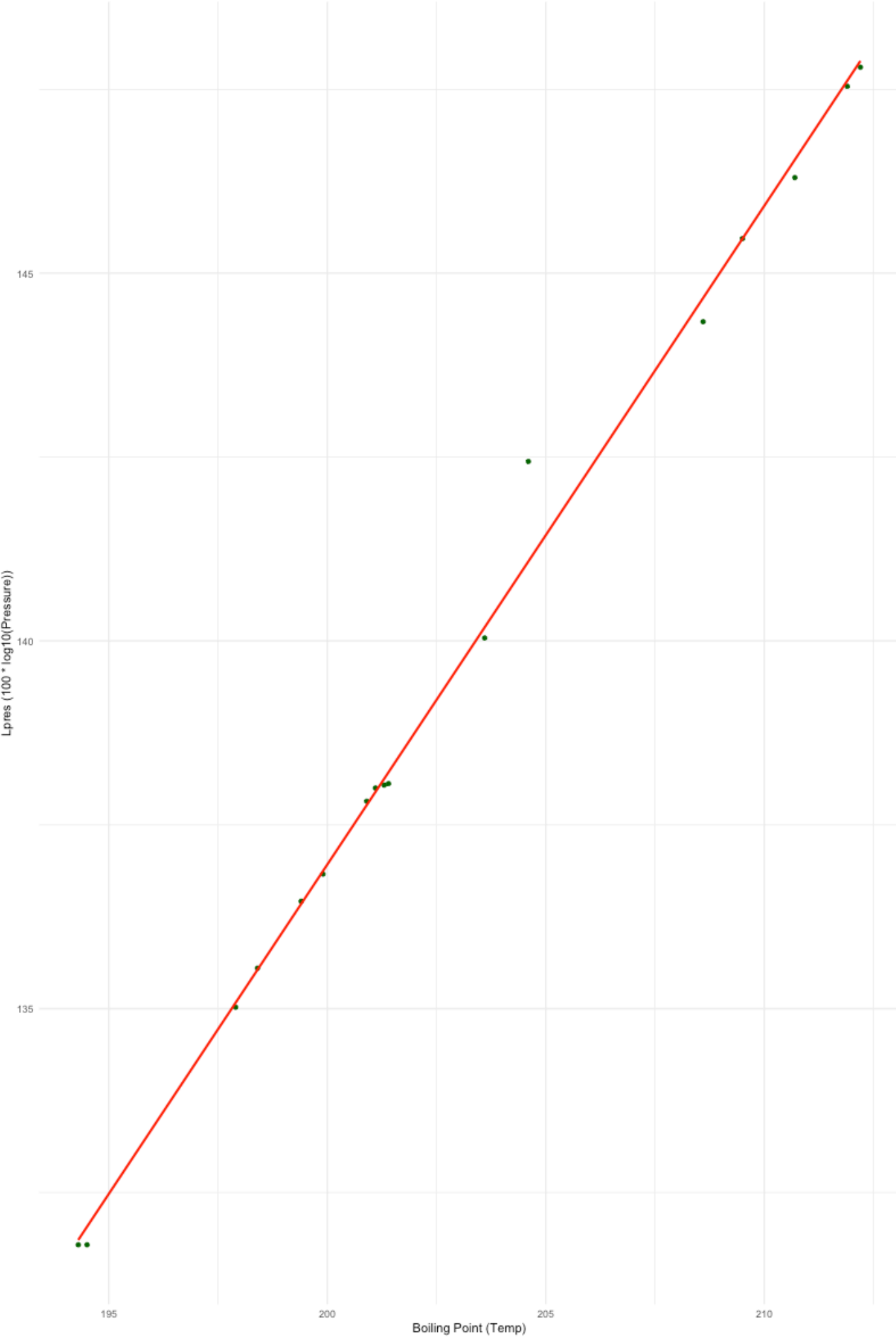
"

Using 'ggplot' in R, add a regression line of the fitted model in the scatter plot between 'Lpres' and 'Temp'.

"

```
ggplot(df, aes(x = Temp, y = Lpres)) +
  geom_point(color = "darkgreen") +
  geom_smooth(method = "lm", se = FALSE, color = "red") + # Adds the regression line
  labs(
    title = "Lpres vs. Boiling Point (Temp) with Fitted Regression Line",
    x = "Boiling Point (Temp)",
    y = "Lpres (100 * log10(Pressure))"
  ) +
  theme_minimal()
```

Lpres vs. Boiling Point (Temp) with Fitted Regression Line



W7Q1e

"

Compute the Residual Sum of Squares (RSS) and
Total sum of squares (TSS) for the above fitted model.

"

Get residuals and fitted values from model_1

residuals_1 <- residuals(model_1)

fitted_1 <- fitted(model_1)

Calculate Residual Sum of Squares (RSS)

rss_1 <- sum(residuals_1^2)

cat("Residual Sum of Squares (RSS):", rss_1, "\n")

Calculate Total Sum of Squares (TSS)

mean_lpres <- mean(df\$Lpres)

tss_1 <- sum((df\$Lpres - mean_lpres)^2)

cat("Total Sum of Squares (TSS):", tss_1, "\n")

```
r$> # Get residuals and fitted values from model_1
residuals_1 <- residuals(model_1)
fitted_1 <- fitted(model_1)

# Calculate Residual Sum of Squares (RSS)
rss_1 <- sum(residuals_1^2)
cat("Residual Sum of Squares (RSS):", rss_1, "\n")

# Calculate Total Sum of Squares (TSS)
mean_lpres <- mean(df$Lpres)
tss_1 <- sum((df$Lpres - mean_lpres)^2)
cat("Total Sum of Squares (TSS):", tss_1, "\n")
Residual Sum of Squares (RSS): 2.154927
Total Sum of Squares (TSS): 427.794
```

W7Q1f

"

Find Residual standard error (RSE) for the above fitted model. And, comment
on the obtained value.

"

n = number of observations, p = number of parameters (beta_0, beta_1)

n_1 <- nrow(df)

p_1 <- 2 # (Intercept and Temp)


```
df_1 <- n_1 - p_1 # Degrees of freedom
```

```
# Calculate RSE
```

```
rse_1 <- sqrt(rss_1 / df_1)
```

```
cat("Residual Standard Error (RSE):", rse_1, "\n")
```

```
# Verifying this using the summary() function
```

```
summary(model_1)$sigma
```

```
comment <- "
```

```
The Residual Standard Error (RSE) is 0.3790275.
```

```
This value represents the average deviation of the observed  
Lpres values from the fitted regression line.
```

```
Given that the Lpres values range from approximately 131 to 148,  
an average error of 0.3790275 is very small,  
indicating that the model fits the data extremely well.
```

```
"
```

```
cat("Residual Standard Error (RSE):", rse_1, "\n")
```

```
# Verifying this using the summary() function
```

```
summary(model_1)$sigma
```

```
comment <- "
```

```
The Residual Standard Error (RSE) is 0.3790275.
```

```
This value represents the average deviation of the observed  
Lpres values from the fitted regression line.
```

```
Given that the Lpres values range from approximately 131 to 148,  
an average error of 0.3790275 is very small,  
indicating that the model fits the data extremely well.
```

```
"
```

```
Residual Standard Error (RSE): 0.3790275
```

```
[1] 0.3790275
```

```
# W7Q1g
```

```
"
```

```
Find R^2 for the above fitted model. And, interpret the value obtained.
```

```
"
```

```
print(summary(model_1)$r.squared)
```

```
comment <- "R^2 value of 0.9949627 means that ~ 99.45% of the  
variability observed in Y ( LPres ) can be explained by X ( Temp )  
using this linear model. This is an exceptionally high value,  
confirming the very strong linear relationship between Lpres and Temp."
```

```

r$> # W7Q1g

"
Find R^2 for the above fitted model. And, interpret the value obtained.
"
print(summary(model_1)$r.squared)

comment <- "R^2 value of 0.9949627 means that ~ 99.45% of the
variability observed in Y ( LPres ) can be explained by X ( Temp )
using this linear model. This is an exceptionally high value,
confirming the very strong linear relationship between Lpres and Temp."
[1] "\nFind R^2 for the above fitted model. And, interpret the value obtained.\n"
[1] 0.9949627

r$>

```

W7Q2a

"

If the model of the above question is modified as follows:

$Lpres = \beta_0 + \beta_1 u_1 + \varepsilon$

where, $u_1 =$

1

Ktemp

And, Ktemp represents the temperature in degrees Kelvin which is given by:

$Ktemp = 255.37 + 5/9 \times Temp$.

Based on the information given, answer the following questions:

"

"

Using the 'ggplot' in R, plot the scatter plot of 'Lpres' versus u_1 and comment on the relationship between two variables.

"

```
if (!require(dplyr)) install.packages("dplyr")
```

```
library(dplyr)
```

```
df <- df %>%
```

```
  mutate(
```

```
    Ktemp = 255.37 + (5 / 9) * Temp,
```

```
    u1 = 1 / Ktemp
```

```
  )
```

```
# Plot Lpres versus u1
```

```
ggplot(df, aes(x = u1, y = Lpres)) +
```

```
  geom_point(color = "purple") +
```

```
  geom_smooth(method = "lm", se = FALSE, color = "red") +
```

```
  labs(
```

```
    title = "Lpres vs. u1 (1 / Ktemp)",
```

```
    x = "u1 (1 / Ktemp)",
```

```
    y = "Lpres"
```

```
  ) +
```

```
  theme_minimal()
```

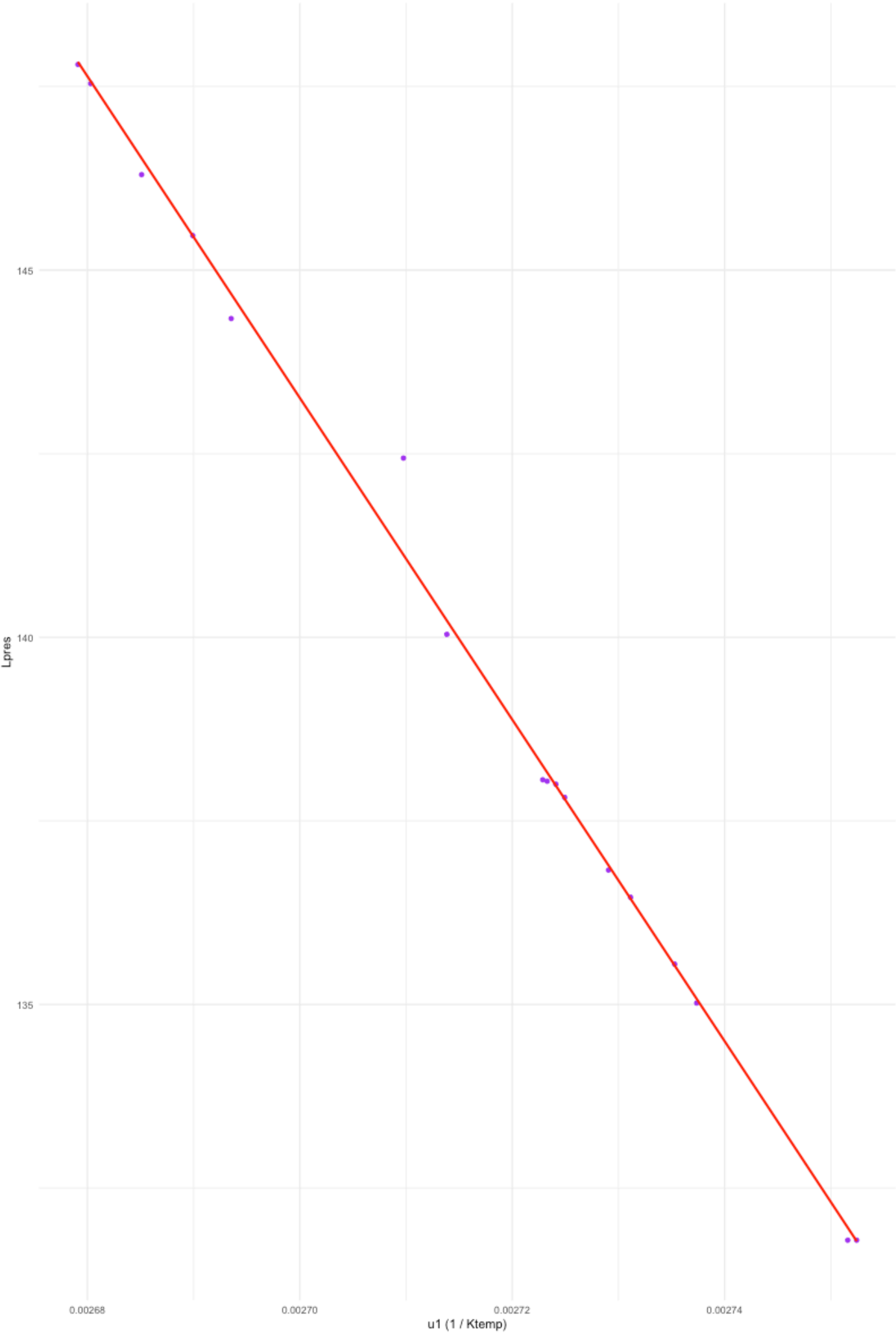
comment <- "

The scatter plot shows a very strong, negative, and linear relationship between u_1 and L_{pres} . As u_1 (the inverse of temperature in Kelvin) increases, L_{pres} decreases linearly.

This transformation appears to have captured the relationship well.

"

Lpres vs. u1 (1 / Ktemp)



```
# W7Q2b
```

```
"
```

```
Fit the linear regression for the given model, and summarize your results.
```

```
"
```

```
model_inv <- lm(Lpres ~ u1, data = df)
```

```
beta_hat_lm_inv <- coef(model_inv)
```

```
cat("\nEstimates of Lpres ~ u1 from lm() function:\n")
```

```
print(beta_hat_lm_inv)
```

```
summary(model_inv)
```

```
summary <- "
```

```
Coefficients:
```

```
Intercept (B0): 734.47 (p-value < 2e-16)
```

```
Slope (B1): -218968.41 (p-value < 2e-16)
```

```
R-squared: 0.9953
```

```
Residual Standard Error (RSE): 0.3673
```

```
Both coefficients are highly statistically significant.
```

```
The R-squared is 0.9953, and the RSE is 0.3673.
```

```
"
```

```

r$> model_inv <- lm(Lpres ~ u1, data = df)
beta_hat_lm_inv <- coef(model_inv)

cat("\nEstimates of Lpres ~ u1 from lm() function:\n")
print(beta_hat_lm_inv)
summary(model_inv)

Estimates of Lpres ~ u1 from lm() function:
(Intercept)          u1
  734.4737 -218968.4109

Call:
lm(formula = Lpres ~ u1, data = df)

Residuals:
    Min       1Q   Median       3Q      Max
-0.33396 -0.17132 -0.03424  0.01791  1.31762

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    734.47      10.59   69.37  <2e-16 ***
u1    -218968.41    3897.33  -56.18  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3673 on 15 degrees of freedom
Multiple R-squared:  0.9953,    Adjusted R-squared:  0.995
F-statistic: 3157 on 1 and 15 DF,  p-value: < 2.2e-16

```

W7Q2c

"

Plot the fitted values obtained from model fitted in 1(c) versus the fitted values from the model 2(b). On the basis of it, commend on if a model can be preferred over the other.

"

Get fitted values from model_2

```
fitted_2 <- fitted(model_inv)
```

Create a data frame of fitted values

```
fitted_df <- data.frame(
  Model_1_Fitted = fitted_1,
  Model_2_Fitted = fitted_2
)
```

Plot fitted values against each other

```
ggplot(fitted_df, aes(x = Model_1_Fitted, y = Model_2_Fitted)) +
  geom_point(color = "orange") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "black") + # y=x line
```

```
labs(  
  title = "Model 2 Fitted Values vs. Model 1 Fitted Values",  
  x = "Fitted Values from Model 1 (Lpres ~ Temp)",  
  y = "Fitted Values from Model 2 (Lpres ~ u1)"  
) +  
theme_minimal()
```

```
comment <- "
```

The plot shows the fitted values from both models are extremely similar,
falling almost perfectly on the $y=x$ dashed line.

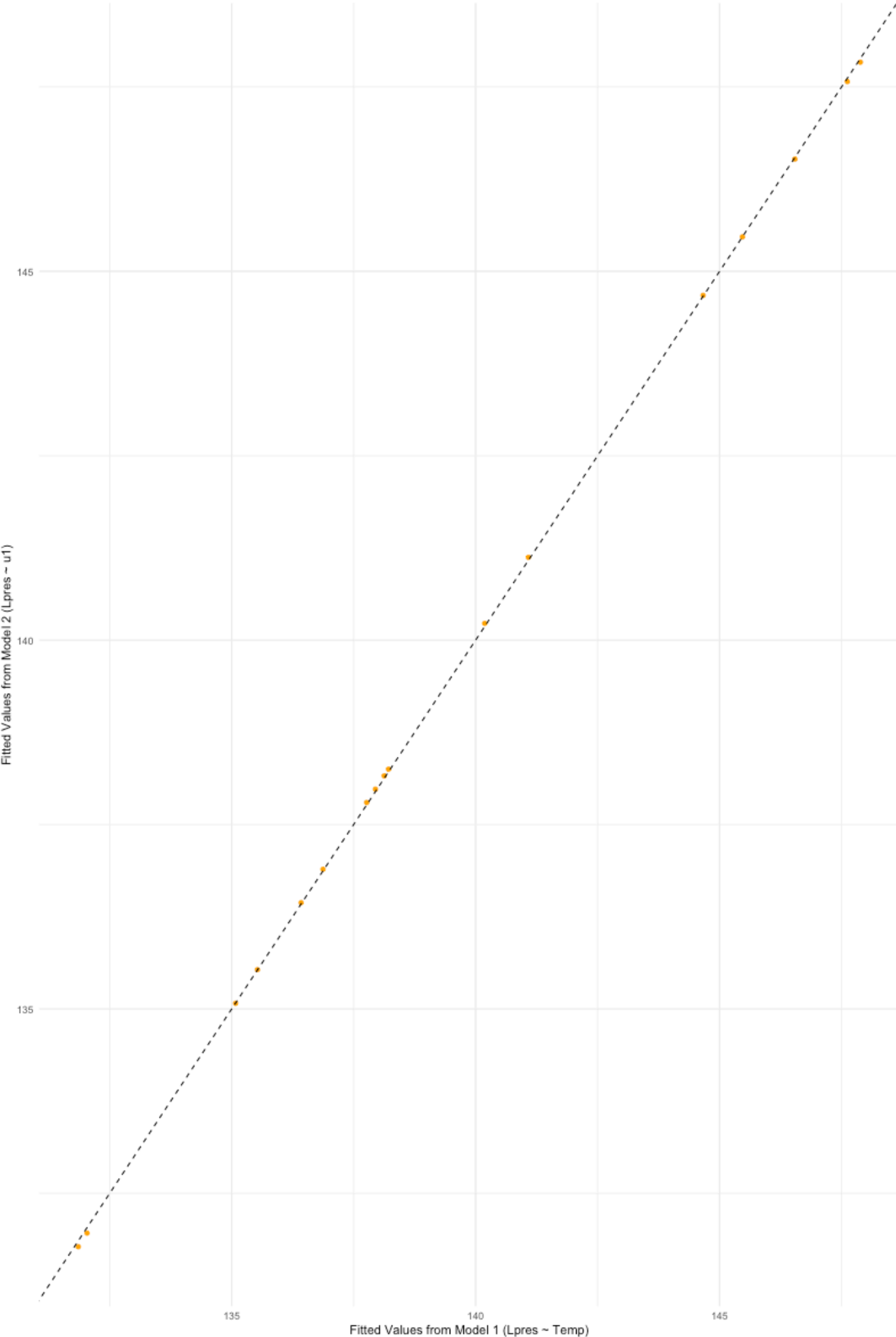
This means both models provide nearly identical predictions.

Model 2 has a slightly higher R-squared (0.995 vs 0.9953)
and a significantly lower RSE (0.379 vs 0.3673).

This indicates a statistically better fit to the data.

```
"
```

Model 2 Fitted Values vs. Model 1 Fitted Values




```
# W7Q3
```

```
"
```

```
Consider a model as
```

$$Y = -1 + 5X + u$$

```
"
```

```
# Set a seed for reproducibility
```

```
set.seed(123)
```

```
# W7Q3a
```

```
"
```

```
Generate 80000 random values from the uniform distribution with parameters
```

```
a = 0 and b = 30 for X.
```

```
"
```

```
X_pop <- runif(80000, min = 0, max = 30)
```

```
X_pop
```

```
# W7Q3b
```

```
"
```

```
Generate 80000 random values from the normal distribution with parameters
```

```
mean = 0 and sd = 20 for u.
```

```
"
```

```
u_pop <- rnorm(80000, mean = 0, sd = 20)
```

```
u_pop
```

```
# W7Q3c
```

```
"
```

```
Using the above generated values, compute the values of Y . And, store the  
respective values of X and Y in a data frame.
```

```
"
```

```
Y_pop <- -1 + 5 * X_pop + u_pop
```

```
Y_pop
```

```
# Store in a data frame
```

```
population_df <- data.frame(X = X_pop, Y = Y_pop)
```

```
head(population_df)
```

```
282 "  
283 Generate 80000 random values from the uniform distribution with parameters  
284 a = 0 and b = 30 for X.  
285 "  
286 X_pop <- runif(80000, min = 0, max = 30)  
287 X_pop  
288  
289 # W7Q3b  
290  
291 "  
292 Generate 80000 random values from the normal distribution with parameters  
293 mean = 0 and sd = 20 for u.  
294 "  
[79871] 4.613706e+00 2.413071e+01 2.157160e+01 2.580318e+01 1.882533e+01 8.874859e+00 2.627543e+01 4.990821e+00 1.869720e+01 1.997256e+01  
[79881] 1.201743e+01 2.558168e+01 2.090319e+01 2.366857e+01 8.055674e+00 4.210420e+00 3.661934e+00 3.962127e+00 1.961063e+01 2.711489e+01  
[79891] 1.938242e+01 8.311565e+00 1.256908e+01 2.444509e+01 2.502355e+00 1.238001e+01 2.626752e+01 2.070251e+01 2.457460e+01 2.244613e+01  
[79901] 2.322034e+01 1.821961e+01 8.980076e+00 2.104198e+01 4.924288e+00 3.315188e+00 2.286733e+01 2.086663e+01 1.700322e+01 2.316439e+01  
[79911] 8.180228e+00 1.098813e+01 2.513696e+01 2.851806e+01 2.437372e+00 2.141318e+01 1.234286e+01 2.229643e+01 1.573288e+01 1.808217e+01  
[79921] 3.961252e+00 1.196408e+01 7.671068e+00 2.360056e+01 2.815933e+00 2.448526e+01 2.025030e+01 2.165517e+01 2.775693e+01 2.940750e+01  
[79931] 7.507702e+00 2.748656e+01 1.188194e+00 2.364493e+01 2.371769e+01 3.674679e+01 2.461527e+01 2.360391e+01 2.935927e+01 1.076238e+01  
[79941] 1.924749e+01 2.019402e+01 2.786917e+00 7.273090e+00 2.030540e+01 1.140815e+01 2.059281e+01 2.844115e+01 2.053320e+01 1.362755e+01  
[79951] 2.518068e+01 1.483404e+01 1.756295e+01 1.719955e+01 2.733699e+01 1.083022e+01 9.258739e+00 2.406738e+01 2.602142e+01 2.024452e+01  
[79961] 3.760892e+00 2.702893e+01 2.553796e+01 1.941954e+01 2.381610e+01 1.543039e+01 6.555480e+00 2.189698e+01 2.713179e+01 1.921696e+01  
[79971] 6.905013e+00 2.088225e+01 8.008292e+00 1.824467e+01 1.257864e+01 2.879158e+01 2.875390e+01 2.299587e+01 2.153613e+01 1.760333e+01  
[79981] 1.355665e+01 1.618301e+01 1.722176e+01 2.822165e+01 2.854605e+01 2.038644e+01 2.246401e+01 2.719608e+01 2.285724e+01 5.778624e+00  
[79991] 6.404754e+00 2.064038e+01 1.070272e+01 1.991519e+01 2.815804e+01 2.675736e+01 1.800734e+01 2.710425e+01 2.403672e+01 1.824395e+01  
r$>
```

```
295  
296 u_pop <- rnorm(80000, mean = 0, sd = 20)  
297 u_pop  
298  
299 # W7Q3c  
300  
301 "  
302 Using the above generated values, compute the values of Y . And, store the  
303 respective values of X and Y in a data frame.  
[79912] -1.264825e-01 2.790878e+01 -2.144950e+01 1.187473e+01 1.442913e+01 4.241125e+01 -9.374781e+00 2.200131e+01 -1.174645e+01  
[79921] -9.353135e+00 8.334695e+00 -1.285464e+01 1.937961e+00 -1.372986e+01 6.910365e+00 3.350457e+01 4.690651e+01 9.211172e+00  
[79930] -3.587626e+01 2.226125e+01 1.368603e+00 2.037649e+01 2.368516e+01 2.672325e+01 -7.193929e+00 1.654680e+00 8.278125e+00  
[79939] 6.420462e+00 3.312080e+01 9.996580e-01 -4.742036e-01 -3.542326e+01 1.870813e+01 7.235906e-01 -1.609228e+01 1.635757e+01  
[79948] -3.114947e+00 1.964576e+01 4.688793e+00 -2.948647e+00 -1.227803e+01 -2.888706e+01 -3.051458e+01 1.008828e+00 -2.494580e+01  
[79957] -9.127758e+00 5.208773e+00 1.804230e+01 -1.268869e+01 2.936659e+01 2.800373e+01 6.945129e+00 1.535053e+01 9.161134e+00  
[79966] 2.951903e+01 1.342444e+01 8.544229e+00 -8.652127e+00 -9.156664e+00 1.089213e+01 -1.949168e+01 8.987851e+00 -2.984083e+01  
[79975] -2.541918e+01 1.332310e+01 -4.367885e+00 -2.838119e+01 9.797578e+00 -7.640163e+00 1.005945e+01 -7.315622e+00 9.769962e+00  
[79984] -1.475321e+01 -8.329975e+00 2.576249e+01 2.865712e+01 2.663511e+01 -2.672348e+01 -7.150545e+00 1.948224e+01 -1.184338e+01  
[79993] 3.078116e+01 -5.688361e-01 3.665223e+01 4.425345e+00 -2.896983e+01 3.079131e+01 1.307218e+01 -5.289416e+01  
r$>
```

```
302 Using the above generated values, compute the values of Y . And, store the  
303 respective values of X and Y in a data frame.  
304 "  
305 |  
306 Y_pop <- -1 + 5 * X_pop + u_pop  
307 Y_pop  
308  
309 # Store in a data frame  
310 population_df <- data.frame(X = X_pop, Y = Y_pop)  
311  
[79912] 53.814148658 152.593590311 120.140773004 23.061594232 120.495054294 103.125560897 101.107388735 99.665710471 77.664396967  
[79921] 9.453127396 67.155087116 24.500696526 118.940768307 -0.650196166 128.336684951 133.756071038 154.182386091 146.995831072  
[79930] 110.161261151 58.799755837 137.801395822 25.317454777 140.909805946 144.311684821 -6.356589302 123.731048406 125.297663697  
[79939] 152.216830278 85.932685370 96.237129584 99.495904181 -22.488678915 54.073580564 101.250582138 39.948461875 118.321637522  
[79948] 138.090778696 121.311784282 71.826553308 121.954764349 60.892146840 57.927674162 54.483154990 136.693785643 28.205303311  
[79957] 36.165935841 124.545676484 147.149406981 87.533906058 47.171053165 162.148383603 133.634931999 111.448239924 127.241649268  
[79966] 105.670974158 45.201837712 117.029119372 126.006826499 85.928113809 44.417192917 83.919552061 48.029313078 60.382517932  
[79975] 36.474044660 156.280982442 138.401600920 85.598168281 116.660401359 79.376499766 76.842708865 72.599450991 94.878766694  
[79984] 125.355016785 133.400283975 126.694706753 139.977149563 161.615496970 86.562736895 20.742575079 50.506011875 90.358517755  
[79993] 83.294752992 98.007099097 176.442430547 137.212148346 -29.069458378 165.312559964 132.255794514 37.325586828  
r$>
```

```

309 # Store in a data frame
310 population_df <- data.frame(X = X_pop, Y = Y_pop)
311
312 head(population_df)
313
314 # W7Q3d
315
316 "
317 Take a sample of size 100 from the created data frame. Using the sampled values
318 of X and Y , fit a linear model, i.e.  $Y = \beta_0 + \beta_1 X + u$ .

```

PROBLEMS 45 OUTPUT TERMINAL PORTS DEBUG CONSOLE

R Interactive + - [] ... [] X

```

r$> population_df <- data.frame(X = X_pop, Y = Y_pop)

head(population_df)
  X      Y
1 0.8138651 -11.12896
2 8.6745082  29.21836
3 29.2180604 123.17356
4 18.5456816  87.55403
5 5.6632483  42.29161
6 8.9273999  15.07320

```

W7Q3d

"

Take a sample of size 100 from the created data frame. Using the sampled values of X and Y , fit a linear model, i.e. $Y = \beta_0 + \beta_1 X + u$.

"

```
sample_df <- population_df[sample(nrow(population_df), 100), ]
```

Fit the linear model

```
model_sample <- lm(Y ~ X, data = sample_df)
```

```
cat("Coefficients from one sample:\n")
```

```
print(coef(model_sample))
```

```

r$> sample_df <- population_df[sample(nrow(population_df), 100), ]

# Fit the linear model
model_sample <- lm(Y ~ X, data = sample_df)

cat("Coefficients from one sample:\n")
print(coef(model_sample))
Coefficients from one sample:
(Intercept)      X
-1.591035      5.189004

r$>

```

W7Q3e

"

Repeat the steps in (d) 8000 times, and store the values of the least square estimates of the coefficients in a matrix.

"

Set up the simulation

```

n_sims <- 8000
sample_size <- 100
estimates_matrix <- matrix(NA, nrow = n_sims, ncol = 2)
colnames(estimates_matrix) <- c("beta_0", "beta_1")

# Run the loop
for (i in 1:n_sims) {
  # Take a sample of size 100
  sim_sample <- population_df[sample(nrow(population_df), sample_size), ]

  # Fit the model
  fit <- lm(Y ~ X, data = sim_sample)

  # Store the coefficients
  estimates_matrix[i, ] <- coef(fit)
}

# Convert to a data frame for easier plotting
estimates_df <- as.data.frame(estimates_matrix)

cat("First 5 sets of estimates:\n")
head(estimates_df)
View(estimates_df)

```

The screenshot shows the RStudio interface. The left pane contains the R script being executed, and the right pane displays the output of the `head(estimates_df)` command. The output is a data frame with 11 rows and 3 columns: (row), beta_0, and beta_1.

(row)	beta_0	beta_1
1	1.4211	5.0027
2	-2.7507	4.9039
3	-0.9469	4.9821
4	-3.8812	4.8409
5	4.5876	4.6352
6	4.2104	4.668
7	-5.3564	5.2744
8	1.0903	5.0405
9	-2.0581	5.2569
10	1.8913	4.9586
11	6.9771	4.6538

W7Q3f

"

Plot the histograms for the values of β^0 obtained in part (e). Also, find the distribution for β^0 and plot the density curve of the

identified distribution on the plotted histogram.

"

```
# Calculate mean and sd for the density curve
```

```
mu_beta0 <- mean(estimates_df$beta_0)
```

```
sd_beta0 <- sd(estimates_df$beta_0)
```

```
cat("Mean of beta_0 estimates:", mu_beta0, "\n")
```

```
cat("SD of beta_0 estimates:", sd_beta0, "\n")
```

```
# Plot histogram and density curve [cite: 60, 61]
```

```
ggplot(estimates_df, aes(x = beta_0)) +
```

```
  geom_histogram(aes(y = ..density..), bins = 50, fill = "lightblue", color = "black") +
```

```
  stat_function(
```

```
    fun = dnorm,
```

```
    args = list(mean = mu_beta0, sd = sd_beta0),
```

```
    color = "red",
```

```
    size = 1
```

```
  ) +
```

```
  labs(
```

```
    title = "Sampling Distribution of Intercept ( $\beta_0$ )",
```

```
    subtitle = paste("True  $\beta_0 = -1$ , Sample Mean =", round(mu_beta0, 4)),
```

```
    x = " $\beta_0$  Estimate",
```

```
    y = "Density"
```

```
  ) +
```

```
  theme_minimal()
```

```
comment <- "
```

```
Distribution for  $\beta_0$ :
```

```
- The histogram clearly shows a symmetric, bell-shaped distribution.
```

```
  The identified distribution is the Normal distribution.
```

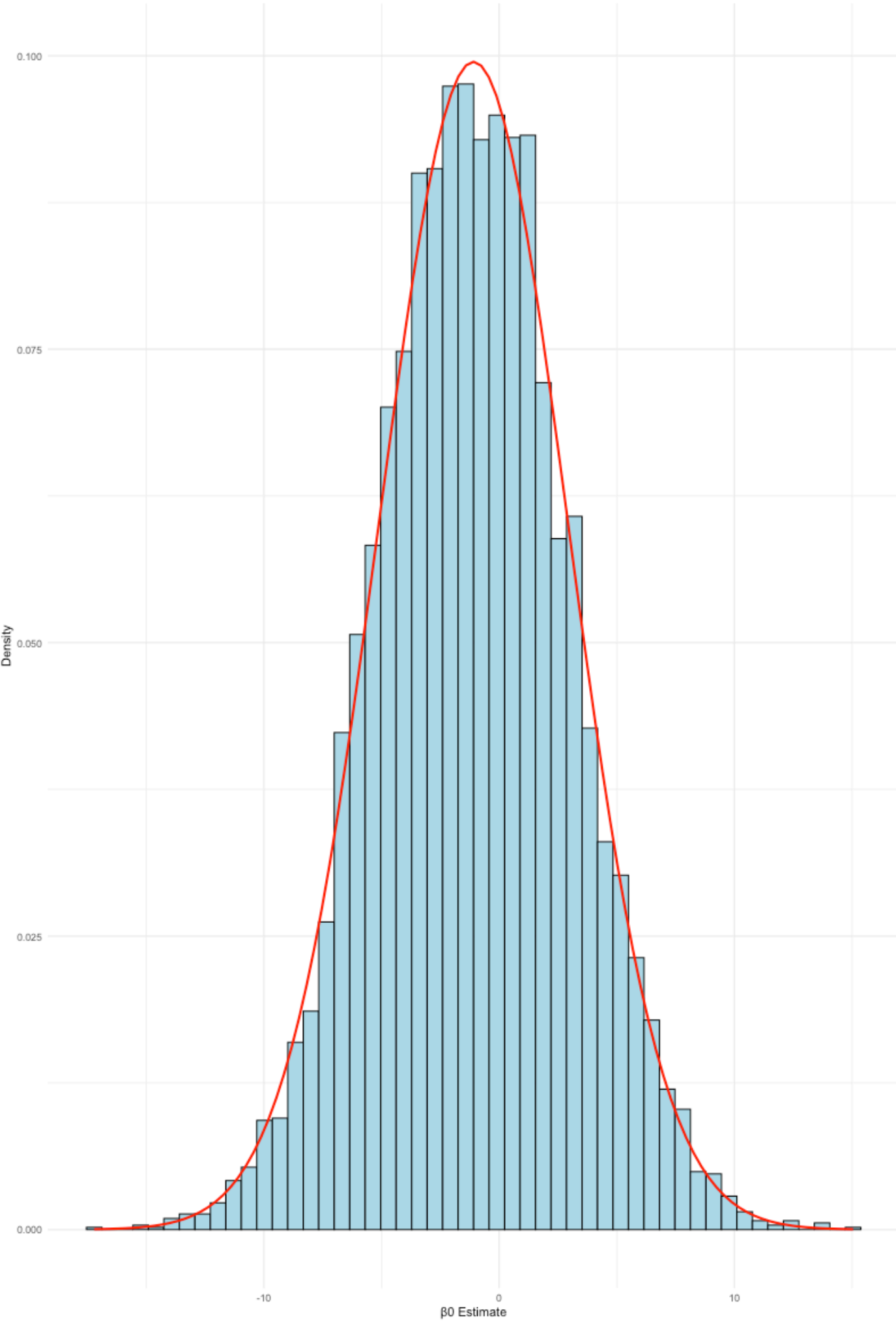
```
- The mean of the 8,000  $\beta_0$  estimates (approx. -1.0)
```

```
is extremely close to the true value of  $\beta_0 = -1$ ,
```

```
demonstrating that the OLS estimator for the intercept is unbiased.
```

```
"
```

Sampling Distribution of Intercept (β_0)
True $\beta_0 = -1$, Sample Mean = -1.0923



```
# W7Q3g
```

```
"
```

Plot the histograms for the values of β^1 obtained in part (e). Also, find the

distribution for β^1 and plot the density curve of the identified distribution on the plotted histogram.

```
"
```

```
# Calculate mean and sd for the density curve
```

```
mu_beta1 <- mean(estimates_df$beta_1)
```

```
sd_beta1 <- sd(estimates_df$beta_1)
```

```
cat("Mean of beta_1 estimates:", mu_beta1, "\n")
```

```
cat("SD of beta_1 estimates:", sd_beta1, "\n")
```

```
# Plot histogram and density curve [cite: 63, 64]
```

```
ggplot(estimates_df, aes(x = beta_1)) +
```

```
  geom_histogram(aes(y = ..density..), bins = 50, fill = "lightgreen", color = "black") +
```

```
  stat_function(
```

```
    fun = dnorm,
```

```
    args = list(mean = mu_beta1, sd = sd_beta1),
```

```
    color = "blue",
```

```
    size = 1
```

```
  ) +
```

```
  labs(
```

```
    title = "Sampling Distribution of Slope ( $\beta_1$ )",
```

```
    subtitle = paste("True  $\beta_1 = 5$ , Sample Mean =", round(mu_beta1, 4)),
```

```
    x = " $\beta_1$  Estimate",
```

```
    y = "Density"
```

```
  ) +
```

```
  theme_minimal()
```

```
comment <- "
```

Distribution for β^1 :

- The histogram clearly shows a symmetric, bell-shaped distribution.

The identified distribution is the Normal distribution.

- The mean of the 8,000 β^1 estimates (approx. 5)

is extremely close to the true value of $\beta_1 = 5$,

demonstrating that the OLS estimator for the intercept is unbiased.

```
"
```

Sampling Distribution of Slope (β_1)
True $\beta_1 = 5$, Sample Mean = 5.0083

